



PensionApp System Documentation

UPS PensionsGo

Current-state master reference covering architecture, operations, data, security, and governance.

Audience	Project stakeholders, engineers, administrators, and support teams
Document version	1.0
Status	Draft for Review
Classification	Internal Use
Repository snapshot date	2026-05-28
Document owner	UPS PensionsGo Project Team
Prepared by	OpenAI Codex

Document Control

Field	Value
Document title	PensionApp System Documentation
Source markdown	PensionApp_System_Documentation.md
Audience	Project stakeholders, engineers, administrators, and support teams
Classification	Internal Use
Version	1.0
Status	Draft for Review
Snapshot date	2026-05-28
Prepared by	OpenAI Codex
Document owner	UPS PensionsGo Project Team

Revision History

Version	Date	Summary	Author	Approval Status
1.1	2026-05-28	Refreshed for the current repository snapshot, schema baseline, and May 2026 documentation rebuild.	OpenAI Codex	Pending

Approval and Sign-Off

Use this page during review and handover to record document acceptance, ownership, and the latest approval state.

Role	Name / Team	Signature	Date
Prepared by	Project Documentation / Engineering		
Reviewed by	Technical Lead / Operations Lead		
Approved by	Project Sponsor / Head of Pensions		

Table of Contents

Note: If the contents do not populate immediately, open the document in Word and update fields.

System: UPS PensionsGo Description: Current-state reference for the implemented pension workflow, registry, claims, payroll, communications, and pensioner-service platform for Uganda Prisons Service. Version Manifest: app_version.json -> 1.0.0 / build 20260401.1 / schema 5.2.2 Repository Snapshot Date: 2026-05-28 Preferred Deliverables: docs/PensionApp_System_Documentation.docx, docs/PensionApp_System_Documentation.pdf

Document Profile

Field	Value
System	UPS PensionsGo
Repository root	c:/xampp/htdocs/PROJECTS/PensionApp
Snapshot date	2026-05-28
Primary technologies	PHP, MySQL/MariaDB, HTML, CSS, JavaScript, PowerShell-based PDF export, PWA service worker
Current scale	41 HTML pages; 50 JavaScript files; 264 PHP API files; 83 database tables
Core schema artifacts	database/schema.sql, database/seed.sql, database/procedures.sql
Versioning artifacts	app_version.json, backend/versioning.php, docs/versioning.md
Documentation toolchain	docs/generate_erd_docs.py, docs/build_documentation_suite.py, docs/export_docx_to_pdf.ps1
Runtime model	PHP/Apache web app with MySQL/MariaDB persistence, background notification worker, and Windows Word COM PDF export for documentation builds

Executive Summary

UPS PensionsGo is no longer just a proposed pension administration platform. It is an implemented, working application that combines retirement-intake management, claims and arrears processing, pension-file registry control, payroll reconciliation, life-certificate compliance, internal workflow routing, public support content, and pensioner self-service inside one role-aware web application.

The current repository reflects a broad operational footprint. The application serves internal staff, administrators, supervisors, pensioners, and selected public users through a single codebase with a PWA shell, a large PHP endpoint surface, and a normalized relational schema. The strongest business-data spine still runs from tb_staffdue to tb_fileregistry, but the system now extends well beyond that original intake-and-registry core into analytics digests, message-storage snapshots, feedback workflows, live chat, podcast/content delivery, configurable notification sounds, pensioner death reporting, data-import/export tooling, and system health diagnostics.

This document is the current-state reference for the implemented platform. It is intended for engineers, administrators, support staff, and project stakeholders who need one dependable description of what exists in the repository today, how the main domains fit together, and which operational controls are already present in code.

1. Current Application Status

The live repository contains a full pension operations platform with the following implemented surfaces:

Surface	Current status
Internal operations workspace	Implemented across dashboards, workflow pages, claims, registry, payroll, messaging, and user-management screens
Administration workspace	Implemented through admin_dashboard.html, settings APIs, system health diagnostics, backup/restore, imports, exports, roles, permissions, and audit views
Pensioner portal	Implemented through pensioner_board.html and pensioner_lookup.html with role-aware controls and consent-driven visibility
Public information surface	Implemented through index.html, about.html, faq.html, feedback.html, terms.html, and podcast_public.html
Progressive Web App shell	Implemented through manifest.webmanifest, service-worker.js, version endpoints, install guidance, and update checks
Documentation suite	Implemented in docs/ with markdown sources, ERD generators, DOCX builds, PPTX decks, and PDF export support

Key implementation notes:

- The front end is static-page based rather than SPA-based. Each page has a focused controller, with shared bootstrapping in frontend/js/main.js.
- The back end is endpoint-oriented. Most actions are exposed as small PHP scripts under backend/api/, with shared runtime behavior concentrated in backend/config.php and domain helper files.
- The schema is maintained centrally in database/schema.sql, while some runtime hardening and table-creation behavior still exists in PHP helper code.
- Documentation generation is now repeatable from the repo: ERDs, Word documents, PDFs, handouts, and presentations can be rebuilt from the current markdown and schema sources.

2. Scope and Functional Coverage

The implemented platform spans the following major domains.

Domain	What is implemented today	Primary pages	Representative APIs	Core tables
Claims and intake	Staff-due capture, application submission, queueing, status updates, delete-request handling, and claims analytics	staff_due.html, application_form.html, application_status.html, claims.html	add_staffdue.php, submit_application.php, get_claims_dashboard.php, update_appn_status.php, request_staff_due_delete.php	tb_staffdue, tb_application_queue, tb_appnstatus, tb_appnsubmissions, tb_claimstatus, tb_staff_due_delete_requests

PensionApp System Documentation

Workflow and tasks	Assignment, delegation, task comments, alerts, completion queue, and workflow logs	dashboard.html, tasks.html	assign_task.php, delegate_task.php, get_task_alerts.php, process_task_completion_queue.php	tb_tasks, tb_task_alerts, tb_task_comments, tb_task_completion_queue, tb_task_delegation_logs, tb_workflow_logs
Registry and documents	Pension-file registry, document uploads, document viewing, file movement, delete queue, recycle bin, box allocation, and life-certificate management	pension_file_registry.html, file_tracking.html, file_movement.html, document_viewer.html, life_certificate.html	fetch_file_registry.php, upload_registry_document.php, record_file_movement.php, request_file_registry_delete.php, restore_file_registry_recycle_item.php, mark_life_certificate_submission.php	tb_fileregistry, tb_staff_documents, tb_file_movements, tb_file_registry_delete_requests, tb_file_registry_recycle_bin, tb_life_certificate_submissions
Payroll and finance	Monthly payroll uploads, payment-register uploads, payroll matching, cycle replacement, suspension uploads, gratuity schedule analysis, retained-payment tracking, budget forecasting, and downloadable import templates	payroll_upload.html, claims.html, budgeting.html, reports.html	upload_payroll_cycle.php, upload_payroll_register.php, manage_payroll_cycle.php, replace_payroll_cycle.php, upload_suspension_arrears.php, upload_gratuity_schedule.php, download_payroll_template.php, post_budget_forecast.php	tb_payroll_upload_cycles, tb_payroll_upload_entries, tb_registry_payroll_monthly_status, tb_suspension_upload_cycles, tb_suspension_upload_entries, tb_gratuity_schedule_cycles, tb_gratuity_schedule_entries, tb_gratuity_schedule_allocations, tb_retained_payments, tb_budgetforecast
Arrears and accountability	Ledger tracking, recent payments, payment allocations, accountability submissions, template-driven uploads, and export-	claims.html	upload_arrears_payments.php, download_arrears_payments_template.php, submit_arrears_accountability.php, get_arrears_beneficiaries.php, export_claims_table.php	tb_arrears_ledger, tb_arrears_payments, tb_arrears_payment_allocations, tb_arrears_accountability_submissions, tb_arrears_accountability_files, tb_arrearstracking

PensionApp System Documentation

	ready arrears analysis			
Messaging and notifications	Direct messages, recipients, attachments, broadcasts, unread tracking, live chat with group/call/presence support, notification sound library, queue processing, notification digests, and storage snapshots	messages.html	send_message.php, get_messages.php, view_message_attachment.php, check_broadcasts.php, live_chat_send.php, live_chat_poll.php, live_chat_call.php, upload_notification_sound.php, run_notification_digest.php, run_message_storage_snapshot.php	tb_messages, tb_message_recipients, tb_message_attachments, tb_broadcast_messages, tb_notification_queue, tb_notification_digest_runs, tb_message_storage_snapshots, tb_user_broadcast_status
Feedback and content	Public/staff/pensioner feedback intake, workflowed review, FAQ management, terms management, podcast publishing, and video-view tracking	feedback.html, faq.html, terms.html, podcast.html, podcast_public.html	submit_feedback.php, update_feedback_submission.php, get_faq_entries.php, get_terms_clauses.php, save_podcast_video.php, record_podcast_view.php	tb_feedback_submissions, tb_feedback_activity, tb_faq_entries, tb_terms_clauses, tb_podcast_videos, tb_podcast_views
Pensioner services	Pensioner dashboard, application progress view, benefits snapshot, compliance view, claims view, lifecycle/documents view, profile update flow, lookup consent, pensioner account administration, and death reporting	pensioner_board.html, pensioner_lookup.html	get_pensioner_dashboard.php, get_pensioner_lookup_context.php, get_pensioner_accounts_admin.php, update_registry_contact_profile.php, update_pensioner_lookup_visibility.php, update_pensioner_password_admin.php, report_pensioner_death.php, search_pensioners.php	tb_users, tb_fileregistry, tb_life_certificate_submissions, tb_arrears_ledger, tb_registry_payroll_monthly_status, tb_pensioner_death_reports
Administr	Users, roles,	admin_dashboard.	get_app_settings.php,	tb_app_settings, tb_roles,

ation and operation s	permissions, settings, public settings, geolocation settings, session controls, active sessions, system health diagnostics, data import/export, backup/restore , analytics digests, storage, logs, cleanup, and app-version management	html, users.html, register_user.html	update_app_settings.php, get_public_settings.php, geolocation_settings.php, get_admin_settings_insights.php, get_active_sessions.php, get_system_status.php, manage_system_health_diagnostics.php, process_data_import.php, run_data_export.php, create_system_backup.php, restore_system_backup.php, update_app_version.php, run_data_cleanup.php	tb_role_permissions, tb_user_permissions, tb_user_settings, tb_user_sessions, tb_session_settings, tb_session_metrics, tb_backup_logs, tb_data_import_runs, tb_data_export_runs, tb_analytics_snapshots, tb_analytics_digest_runs, tb_system_logs, tb_system_log_resolutions, tb_audit_logs, tb_user_logs, tb_file_scan_logs, tb_ip_geolocation
-----------------------	--	--------------------------------------	--	---

3. Repository and Runtime Architecture

3.1 Repository Map

Area	Responsibility
frontend/	Browser pages, CSS, page controllers, shared JS modules, PWA shell, and static assets
backend/	PHP bootstrap, API endpoints, helper libraries, uploads, workers, diagnostics, and runtime admin tools
database/	Maintained schema baseline, seed/reference data, and reserved stored-routine bundle
docs/	Markdown sources, ERD generator, document builders, handout builder, presentation deck builder, and generated outputs
test/	Current automated-test placeholder area
logs/	Runtime and generated log targets

3.2 Critical Source Files

Path	Why it matters
backend/config.php	Central bootstrap for DB connection, settings, RBAC helpers, schema ensure-functions, audit helpers, logging, and shared business utilities
backend/runtime_admin_tools.php	Runtime helpers for message snapshots, file scanning, analytics digests, and admin operations
backend/system_health_tools.php	System health diagnostics, unresolved incident grouping, incident-resolution logging, and subsystem-specific recommendations
backend/api/SessionManager.php	Session creation, validation, conflict handling, and activity logging
backend/api/TimeoutManager.php	Inactivity timeout, grace period, and admin re-

	authentication timing
frontend/js/main.js	Shared front-end bootstrap, authenticated fetch wrapper, CSRF/device-token flow, viewer routing, and common UI setup
frontend/js/modules/pwa.js	PWA install/update management, version checks, offline banner, and app-shell lifecycle handling
frontend/service-worker.js	Cache namespace handling, install/update lifecycle, and offline fallback behavior
backend/versioning.php	Reads app_version.json, computes cache-safe build fingerprints, and exposes version metadata to the front end
database/schema.sql	Best single-file representation of the current data model

3.3 Architecture Layers

Layer	Implementation pattern
Presentation	Static HTML pages enhanced by page-specific controllers and shared modules
Shared front-end shell	main.js, header/footer loaders, feedback helpers, and PWA/version utilities
API layer	Focused PHP endpoints under backend/api/ returning JSON or file responses
Shared runtime	backend/config.php, runtime helper libraries, export helpers, and diagnostics utilities
Persistence	MySQL/MariaDB relational schema with 83 maintained tables
Background processing	Notification queue worker plus admin-triggered digest/export/cleanup flows
File storage	Uploaded registry documents, message attachments, payroll artifacts, suspensions, profile photos, and accountability files under backend/uploads/

4. Front-End Inventory and User Surfaces

4.1 Page Inventory by Group

Group	Pages
Public and entry	index.html, about.html, faq.html, feedback.html, terms.html, login.html, offline.html, podcast_public.html
Operational dashboards	dashboard.html, admin_dashboard.html, reports.html
Claims and intake	add_staff.html, staff_due.html, view_staff.html, edit_staff.html, claim_form.html, claims.html, application_form.html, application_status.html
Registry and records	pension_file_registry.html, file_registry.html, file_tracking.html, file_movement.html, document_viewer.html, life_certificate.html
Payroll and finance	payroll_upload.html, budgeting.html, benefits_calculator.html
Collaboration and identity	messages.html, profile.html, users.html, register_user.html, edit_user.html, tasks.html

Pensioner services	pensioner_board.html, pensioner_lookup.html
Content and media	podcast.html
Shared partials	header1.html, header2.html, footer.html, footer1.html

4.2 UX and PWA Notes

- The application supports installation as a PWA and includes manifest icons, offline fallback, version-aware update prompts, and a service-worker cache namespace.
- The dashboard has evolved into a broad operational cockpit with claims, demographics, life-certificate exports, payroll upload shortcuts, feedback management, data-management grids, pensioner death reporting, notification controls, public-content settings, and executive reporting.
- The claims page is now a rich operational workspace rather than a simple table. It includes arrears exports, payment uploads, suspension uploads, gratuity schedule analysis, strategic estate/full-pension views, and accountability handling.
- The pensioner dashboard exposes six structured views: profile, application progress, benefits, payroll/compliance, claims, and lifecycle/documents.

5. Back-End Services and Operational Controls

The API surface is broad, but it is consistent in style: each endpoint validates session state, checks authorization, normalizes inputs, calls shared helpers, and returns structured JSON or file responses.

Representative back-end capability areas:

- Authentication and session governance: login, logout, active sessions, timeout settings, concurrent-session controls, device-aware session cleanup, geolocation-aware logging, and admin password verification.
- Role and permission governance: role catalog, permission overrides, current-permission introspection, and cloned-role behavior.
- Registry governance: delete requests, recycle-bin restore/purge, document uploads, file movement logging, and box allocation summary.
- Claims and finance: payroll cycle uploads, payment-register attachments, suspension uploads, gratuity schedules, arrears payments, accountability files, and budget exports.
- Diagnostics and operations: analytics digests, message-storage snapshots, notification queue processing, live-chat attachment storage, storage usage, audit logs, system status, admin settings insights, data cleanup, and backup/restore workflows.
- Security-oriented operations: file scan logs, configurable attachment/document rules, optional ClamAV integration, and system-log incident resolution.

6. Data Architecture and Schema Baseline

The current maintained schema defines 83 tables. The schema is normalized by domain and now includes a dedicated `tb_system_log_resolutions` table so the system health dashboard can persist acknowledgment and resolution actions for warnings and errors raised from `tb_system_logs`. It also includes `tb_pensioner_death_reports` to preserve pensioner lifecycle notifications separately from registry, payroll, and arrears records.

6.1 Anchor Tables

Anchor table	Why it matters
tb_users	Identity anchor for staff and pensioner accounts; referenced across workflow, messaging, feedback, payroll audits, sessions, permissions, and governance flows
tb_staffdue	Retirement-intake record and source of many pre-registry workflow actions
tb_fileregistry	Canonical pension file record that links registry, payroll, life-certificate, claims, and pensioner self-service workflows
tb_tasks	Assignment, routing, comments, alerts, and completion processing spine
tb_payroll_upload_cycles	Monthly payroll reconciliation header
tb_arrears_ledger / tb_arrears_payments	Financial accountability spine for claims and arrears
tb_pensioner_death_reports	Pensioner lifecycle-reporting table for death notifications and follow-up governance
tb_system_logs / tb_system_log_resolutions	Operational diagnostics history and incident-resolution tracking

6.2 Schema Artifacts

Artifact	Purpose
database/schema.sql	Maintained schema baseline for current tables, indexes, auto-increment definitions, and foreign keys
database/seed.sql	Roles, settings, titles, districts, FAQ/terms-related reference data, holidays, and other non-transactional seed content
database/procedures.sql	Reserved routine bundle; currently intentionally a no-op because persistent stored procedures are not required by the app
docs/erd.md / docs/ERD.pdf	Full-schema reference generated from database/schema.sql
docs/erd-domains/*.md	Domain-specific ERD breakdowns

6.3 ERD Strategy

- Use docs/erd.md or docs/ERD.pdf for the full schema.
- Use docs/erd-domains/registry.md, workflow.md, payroll.md, arrears.md, messaging.md, users_access.md, and analytics_ops.md for targeted domain work.
- Use docs/erd-domains/interdomain_links.md and interdomain_matrix.md to understand cross-domain join paths.
- Treat regNo, userId, task IDs, payroll cycle IDs, and arrears payment/ledger IDs as the main system-spanning anchors.

7. Security, Access Control, and Governance

Security is implemented through layered controls rather than one feature.

Control area	Current implementation
Authentication	Session-based PHP auth with login throttling,

	maintenance mode handling, pensioner-login gating, and password expiry support
Session governance	SessionManager, TimeoutManager, device-awareness, concurrent-session rules, active-session inspection, and termination endpoints
Role and permission model	tb_roles, tb_role_permissions, tb_user_permissions, effective-role normalization, and role-cloning support
Request hardening	CSRF token endpoint, authenticated fetch wrapper, optional origin validation, and sensitive-action re-authentication
File governance	Attachment/document allowed types, size rules, dedupe flags, retention settings, and scan logging
Deletion governance	Separate staff-due delete queue plus registry recycle-bin model with restore and purge flows
Public-service governance	Public settings, FAQ/terms management, podcast moderation, lookup-consent controls, and pensioner-facing visibility switches
Notification governance	Broadcast visibility, notification queue processing, user notification sounds, digest runs, and live-chat presence/call state
Diagnostics governance	System logs, audit logs, user logs, workflow logs, task delegation logs, backup logs, and system health incident resolutions

Important policy pairings already enforced in code:

- Pensioner access can be globally disabled through settings.
- Pensioner lookup is governed by consent rather than open visibility.
- Certain pensioner profile fields are hidden and rejected when the underlying record is in a restricted lifecycle state.
- Data Management access is limited to administrators and OC/Pension-equivalent leadership roles.
- Pensioner death reports are captured as governed lifecycle events so follow-up can be audited without overwriting core registry history.

8. Operations and Deployment

8.1 Deployment Model

Area	Current practice
Web runtime	PHP/Apache deployment, with local XAMPP-friendly paths and cPanel-oriented config examples
Environment overrides	backend/config.local.php based on backend/config.local.example.php
Version management	app_version.json plus backend/versioning.php and PWA version endpoints
Email transport	SMTP or PHP mail, with notification queue settings and worker support
Background work	backend/workers/process_notification_queue.php plus admin-triggered digest/export/cleanup routines
Backup and recovery	Admin-triggered system backups, data-artifact downloads, restore workflows, and cleanup tooling

PDF generation for documentation	Windows Word COM automation via docs/export_docx_to_pdf.ps1
----------------------------------	---

8.2 Upload Layout

Current directories under backend/uploads/:

- accountability_forms
- documents
- live_chat
- messages
- payroll
- payrolls
- profiles
- suspensions

9. Documentation and Build Artifacts

The repo now supports a rebuildable documentation set rather than one-off generated files.

Artifact family	Source	Generated outputs
System, technical, user, concept, and proposal docs	Markdown in docs/*.md	DOCX and PDF via docs/build_documentation_suite.py
ERD pack	database/schema.sql -> docs/generate_erd_docs.py	erd.md, erd.mmd, domain ERDs, ERD.docx, ERD.pdf
Retirement formula handout	docs/build_retirement_type_handout.py and reference markdown	Retirement_Type_Formula_Handout.docx, Retirement_Type_Formula_Handout.pdf
Presentation decks	docs/build_presentation_decks.py	PensionApp_Technical_Team_Presentation.pptx, PensionApp_Users_Presentation.pptx

Recommended regeneration flow:

1. Update database/schema.sql and markdown sources.
2. Run python docs/build_documentation_suite.py.
3. Review regenerated .docx, .pdf, .pptx, and ERD outputs under docs/.

10. Current Risks and Recommended Next Steps

Observation	Why it matters	Recommended next step
backend/config.php remains a large shared bootstrap	High coupling and merge risk	Split RBAC, settings, logging, and schema utilities into smaller service modules
Runtime schema hardening still exists in PHP	Can mask release drift	Move toward explicit migrations while keeping light defensive checks
API surface is large	Discoverability and change safety are harder	Publish an API catalogue or OpenAPI baseline for high-value endpoints
Automated tests are minimal	Regression risk is high in a broad operational system	Prioritize tests around auth, registry lifecycle, payroll uploads,

		arrears, and pensioner restrictions
Some legacy query paths still exist	Older table names and compatibility code can confuse maintainers	Continue retiring stale aliases and align docs/routes with the maintained schema

Appendix A. Key Front-End Controllers

Controller	Main area
frontend/js/dashboard.js	Operational dashboard, life certificates, payroll drill-downs, data-management modals, feedback workflows, demographics, and summary reporting
frontend/js/admin-dashboard.js	Admin console, system settings, system health diagnostics, role management, public settings, and operational controls
frontend/js/claims.js	Claims dashboard, exports, payments, accountability, suspension uploads, and gratuity schedule analysis
frontend/js/pension_file_registry.js	Registry browse/edit flows, linked document actions, and record governance
frontend/js/payroll_upload.js	Payroll cycle history, replacement flows, and payment-register handling
frontend/js/pensioner_board.js	Pensioner dashboard, detailed tabs, and profile-update workflow
frontend/js/messages.js	Direct/broadcast messaging, attachments, unread state, and thread views
frontend/js/staff_due.js	Intake records, workflow routing, and delete-request initiation
frontend/js/modules/live_chat.js	Live chat bootstrap, polling, attachments, voice notes, group chat actions, call signalling, and presence indicators
frontend/js/modules/session-worker.js	Browser-side session heartbeat and timeout coordination

Appendix B. Key Back-End Modules

Path	Role
backend/api/data_management_common.php	Exportable data-management datasets and grid support
backend/api/task_workflow_common.php	Shared task/workflow logic
backend/api/import_common.php	Shared import helpers and row-validation support
backend/api/registry_document_common.php	Shared registry-document handling and validation
backend/api/gratuity_schedule_common.php	Shared gratuity schedule analysis helpers
backend/api/live_chat_common.php	Shared live-chat validation, thread/group helpers, attachment handling, and polling support
backend/lib/pdf_library.php	TCPDF-backed PDF export support for operational reports
backend/notification_sound_library.php	Notification sound catalog, upload metadata, and validation helpers
backend/system_health_tools.php	System health alert building and incident resolution
backend/runtime_admin_tools.php	Snapshots, scans, digests, and runtime admin jobs

Appendix C. Maintained Documentation Artifacts

Artifact	Purpose
docs/PensionApp_System_Documentation.md	Current-state master reference
docs/PensionApp_Technical_Documentation.md	Technical copy aligned to the master reference
docs/PensionApp_User_Admin_Manual.md	Operational usage guide
docs/PensionApp_Concept_Paper.md	Current-state concept narrative
docs/PensionApp_Project_Proposal.md	Stabilization and next-phase proposal
docs/erd.md	Full-schema ERD reference
docs/erd-domains/	Domain ERDs and inter-domain views
docs/DFDs/	Data-flow diagram pack
docs/versioning.md	Version manifest and release metadata reference